

Day 46: Deposit Method

Day 46 — Deposit Method

1. Day Number

Day 46

2. Topic Name

Object Method with Validation

Today you will learn how an object method can:

- receive some data
 - check whether the data is valid
 - update an object's attribute
-

3. Connection

Yesterday, you added a method to your `BankAccount` class to **display account details**.

Conceptually, your class has reached this stage:

```
BankAccount
|
|— account_holder
|— balance
|
|— display_account()
```

Today, you will add some **behavior**:

```
BankAccount
|
|— account_holder
|— balance
|
|— display_account()
|
|— deposit()
```

The `deposit()` method will allow money to be added to the account.

4. Important Topics

Today's important concepts are:

- method
- method parameter
- `self`
- object attributes

- attribute update
- if condition
- validation

The important idea is:

```
Receive deposit amount
    ↓
Check whether amount is valid
    ↓
If amount > 0
    ↓
Update balance
```

5. Foundational Notes

What is a method?

A **method** is a function that belongs to a class.

Example structure:

```
class Student:
    def show_name(self):
        print(self.name)
```

Here:

```
show_name()
```

is a method of the Student class.

Methods can receive values

A method can receive additional information through parameters.

For example:

```
def add_score(self, score):
```

Here:

- self represents the current object
- score is the value being passed to the method

The method might be called like:

```
student.add_score(10)
```

The value:

```
10
```

goes into:

```
score
```

What does self do?

self gives the method access to the current object's data.

Suppose an object has:

```
balance = 1000
```

Inside a method, you can access it using:

```
self.balance
```

So:

```
self.balance
```

means:

| the balance belonging to this particular account object.

Updating an attribute

Suppose:

```
balance = 1000  
deposit = 500
```

The new balance should become:

```
1000 + 500 = 1500
```

Conceptually:

```
balance = balance + deposit
```

For an object attribute, the same idea becomes:

```
self.balance = self.balance + amount
```

Python also provides the shorter form:

```
self.balance += amount
```

Both represent the same basic idea.

Why do we need validation?

Imagine someone tries:

Deposit = -500

If you blindly add it:

$1000 + (-500)$

the balance becomes:

500

But a **deposit** should not reduce the balance.

Therefore, before changing the balance, check:

Is amount greater than 0?

This is called **validation**.

Deposit rule

For today's problem:

`amount > 0`

means:

Valid deposit

But:

`amount == 0`

or:

`amount < 0`

means:

Invalid deposit

6. Easy Example

Consider a simple game player:

```
class Player:
    def __init__(self, points):
        self.points = points

    def add_points(self, points):
```

```
if points > 0:  
    self.points += points
```

Suppose:

```
Starting points = 10  
Added points = 5
```

Then:

```
New points = 15
```

The important pattern is:

```
method receives value  
    ↓  
if value is valid  
    ↓  
update object attribute
```

Your BankAccount problem follows the same pattern.

7. Problem Statement

Create or continue your existing BankAccount class.

Add a method named:

```
deposit()
```

The method should receive a deposit amount.

Requirement

If:

```
amount > 0
```

add the amount to the account balance.

If the amount is:

```
0
```

or negative:

```
amount < 0
```

do not add it to the balance.

Example scenario

Initial account:

Account Holder: Rahul
Balance: 1000

Deposit:

500

New balance:

1500

8. Concepts Used

You will use:

Class

Represents the BankAccount.

BankAccount

Object

Represents one actual bank account.

account1

Attribute

Stores account information.

account_holder
balance

Method

Represents account behavior.

deposit()

Parameter

Receives the deposit amount.

amount

self

Allows the method to access the object's balance.

self.balance

if

Checks whether the deposit is valid.

```
if amount is positive
```

Attribute update

Changes the account balance.

```
old balance + deposit  
    ↓  
new balance
```

9. Thought Process

Before writing Python, think through the problem.

Step 1: What object are we working with?

A:

```
BankAccount
```

Step 2: What information does it already have?

For example:

```
account_holder  
balance
```

Step 3: What new behavior do we need?

We need:

```
deposit money
```

So we need a method:

```
deposit
```

Step 4: What information should the method receive?

It needs:

```
deposit amount
```

So conceptually:

```
deposit(amount)
```

Step 5: Should every amount be accepted?

No.

The deposit should only happen when:

`amount > 0`

Step 6: What happens for a valid amount?

Add it to the current balance:

`new balance = current balance + amount`

Step 7: What happens for invalid amounts?

For today's simple version:

`do not change the balance`

You may also display a simple message such as:

`Invalid deposit amount`

if you choose.

10. Pseudocode

```
CREATE BankAccount class

    CREATE constructor
        STORE account holder
        STORE balance

    CREATE deposit method that receives amount

        IF amount is greater than 0
            ADD amount to current balance

        ELSE
            DO NOT change balance

CREATE one BankAccount object

DISPLAY starting balance

CALL deposit method with an amount

DISPLAY updated balance
```


The main logic is:

```
deposit(amount)
  ↓
amount > 0?
  /   \
Yes    No
  ↓     ↓
Add to  Keep balance
balance unchanged
```

11. Suggested Solving Approach

Use an **OOP-based approach**.

Keep the behavior inside the class.

Instead of doing something like:

```
account balance
  ↓
external code changes balance
```

prefer:

```
BankAccount object
  ↓
deposit()
  ↓
object updates its own balance
```

This is one of the important ideas behind OOP:

```
Data + behavior
  ↓
kept together
```

For example:

```
BankAccount
├─ account_holder
├─ balance
├─ display_account()
└─ deposit()
```

12. Easy Edge Cases

Edge Case 1: Deposit 0

Example:

Current balance = 1000
Deposit = 0

Because:

0 is not greater than 0

the balance should remain:

1000

Edge Case 2: Negative Deposit

Example:

Current balance = 1000
Deposit = -200

Because:

-200 < 0

the deposit should not happen.

Balance stays:

1000

Edge Case 3: Normal Positive Deposit

Current balance = 1000
Deposit = 300

Result:

1300

13. Expected Input

For example, assume the account starts with:

Account Holder: Rahul
Balance: 1000

Then call the deposit method with:

500

Conceptually:

```
account1.deposit(500)
```

Another test could be:

```
account1.deposit(0)
```

or:

```
account1.deposit(-100)
```

14. Expected Output

For a valid deposit:

```
Initial Balance: 1000
Deposit Amount: 500
Updated Balance: 1500
```

For deposit 0:

```
Initial Balance: 1000
Deposit Amount: 0
Updated Balance: 1000
```

For a negative deposit:

```
Initial Balance: 1000
Deposit Amount: -200
Updated Balance: 1000
```

The exact wording of your printed output can be different. The important requirement is that invalid deposits **must not change the balance**.

15. Hint Only

Start from yesterday's BankAccount class.

Add a new method that looks conceptually like:

```
def deposit(self, amount):
    # check whether amount is positive

    # if valid:
        # update self.balance
```

Remember these two ideas:

```
if amount > 0:
```

and:

```
self.balance += amount
```

Your job is to combine them correctly inside the `deposit()` method.

Do not forget: the method belongs inside the `BankAccount` class, and you call it through an object:

```
account object  
  ↓  
.deposit(...)  
  ↓  
balance changes
```